MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

Mini Project Report
of
Database Systems Lab (CSS 2212)

Simple Course Management System

SUBMITTED
BY

Name	Registration No.	Roll No.	Section	Branch
Devarjya Purkayastha	240968040	09	A1	Data Science and Engineering

School of Computer Engineering
Manipal Institute of Technology, Manipal.

MANIPAL INSTITUTE OF TECHNOLOGY
MANIPAL
(A constituent unit of MAHE, Manipal)

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

MANIPAL
07/04/2026

CERTIFICATE

This is to certify that the project titled Simple Course Management System is a record of the bonafide work done by Student(s) (Reg. No. 240968040) submitted in partial fulfilment of the requirements for the award of the Degree of Bachelor of Technology (B.Tech.) in DATA SCIENCE AND ENGINEERING of Manipal Institute of Technology, Manipal, Karnataka, (A Constituent Institute of Manipal Academy of Higher Education), during the academic year 2025-2026.

Name and Signature of Examiners:

Dr. Girija Attigeri, Associate Professor, SCE.

Dr. Radhakrishna Bhat , Associate Professor, SCE.

Contents

Abstract **2**

1 Problem Statement & Objectives **3**

2 Entity-Relationship Diagram **4**

3 Relational Schema **6**
3.1 Users 6
3.2 Courses 6
3.3 Enrollments 6
3.4 Assignments 6
3.5 Submissions 7
3.6 Grades 7
3.7 Grade_Audit 7

4 Normalization **8**

5 SQL Components **9**
5.1 Basic Queries (`03_basic_queries.sql`) 9
5.2 Complex Queries (`04_complex_queries.sql`) 9
5.3 Stored Procedures & Functions (`05_procedures_functions.sql`) 10
5.4 Cursors 10
5.5 Triggers (`06_triggers.sql`) 11

6 System Architecture **12**

7 Technologies Used **13**

8 Sample Data Summary **14**

9 Conclusion, Limitations and Future Work **15**

10 References **16**

Abstract

This project aims to develop a simplified **Course Management System (CMS)** that provides core functionalities for assignment submission and grading in a university environment. The system facilitates interaction between students and instructors through a simple interface for managing courses, assignments, and grades. The application follows a three-tier client-server architecture with **MySQL 8.0** as the backend database, **Express.js** for the REST API layer, and **React** for the frontend interface.

This project is based on **Project 9.3** from *Database System Concepts* by Silberschatz, Korth, and Sudarshan (7th Edition).

Problem Statement & Objectives

Source: Project 9.3, *Database System Concepts* — Silberschatz, Korth, Sudarshan (7th Edition)

Universities widely use course-management systems such as Moodle, Blackboard, or WebCT to manage academic workflows digitally. This project implements a representative subset of such a system, focusing on the assignment submission and grading lifecycle. The system provides mechanisms for students and instructors to manage coursework, view grades, and track course progress.

Functional Requirements

- **User Management:** Secure login for students and instructors; role-based access control
- **Course Management:** Create and manage courses; enroll students in courses
- **Assignment Module:** Create assignments with deadlines; submit assignments; view submission history
- **Grading System:** Grade student submissions with marks and feedback; view grades
- **Data Integrity:** Prevent duplicate enrollments, late submissions, and marks exceeding the maximum
- **Audit Trail:** Log all grade changes (insert, update, delete) via triggers

Objectives

- Design and implement a normalised relational database schema for a course management system
- Develop a simple web-based user interface for system interaction
- Enable end-to-end assignment submission and grading workflows
- Demonstrate practical use of SQL features: joins, subqueries, views, stored procedures, cursors, and triggers

Entity-Relationship Diagram

The system consists of six entities with the following relationships:

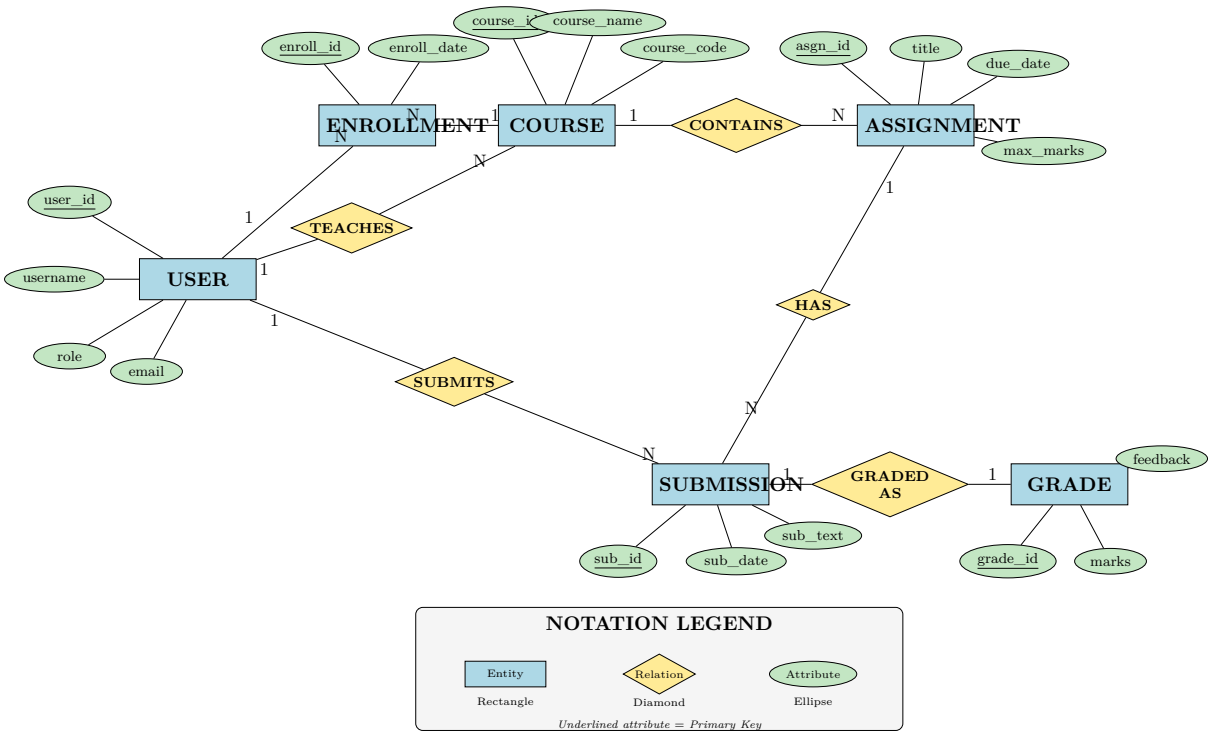


Figure 1: Entity-Relationship Diagram for the Simple Course Management System

Relationship	Cardinality	Description
User (instructor) — Course	1 : N	One instructor can teach many courses.
User (student) — Course (via Enrollment)	M : N	One student can enroll in many courses; one course can have many students. Resolved via the Enrollment junction entity.
Course — Assignment	1 : N	One course can have many assignments.
User (student) — Submission	1 : N	One student can make many submissions across different assignments.
Assignment — Submission	1 : N	One assignment can receive many submissions (one per enrolled student).
Submission — Grade	1 : 1	Each submission receives at most one grade record.

Relational Schema

Users

Column	Type	Constraints
user_id	INT	PRIMARY KEY, AUTO_INCREMENT
username	VARCHAR(50)	NOT NULL, UNIQUE
password	VARCHAR(255)	NOT NULL
role	VARCHAR(20)	NOT NULL, CHECK IN (`'student'`, `'instructor'`)
name	VARCHAR(100)	NOT NULL
email	VARCHAR(100)	NOT NULL, UNIQUE

Courses

Column	Type	Constraints
course_id	INT	PRIMARY KEY, AUTO_INCREMENT
course_name	VARCHAR(100)	NOT NULL
course_code	VARCHAR(20)	NOT NULL, UNIQUE
instructor_id	INT	NOT NULL, FK $\rightarrow$ Users(user_id)

Enrollments

Column	Type	Constraints
enrollment_id	INT	PRIMARY KEY, AUTO_INCREMENT
course_id	INT	NOT NULL, FK $\rightarrow$ Courses(course_id)
student_id	INT	NOT NULL, FK $\rightarrow$ Users(user_id)
enrollment_date	DATE	NOT NULL, DEFAULT CURRENT_DATE
		UNIQUE(course_id, student_id)

Assignments

Column	Type	Constraints
assignment_id	INT	PRIMARY KEY, AUTO_INCREMENT
course_id	INT	NOT NULL, FK $\rightarrow$ Courses(course_id)
title	VARCHAR(200)	NOT NULL
description	TEXT	—
due_date	DATETIME	NOT NULL
max_marks	INT	NOT NULL, CHECK ≥ 1

Submissions

Column	Type	Constraints
submission_id	INT	PRIMARY KEY, AUTO_INCREMENT
assignment_id	INT	NOT NULL, FK → Assignments(assignment_id)
student_id	INT	NOT NULL, FK → Users(user_id)
submission_text	TEXT	NOT NULL
submission_date	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP
		UNIQUE(assignment_id, student_id)

Grades

Column	Type	Constraints
grade_id	INT	PRIMARY KEY, AUTO_INCREMENT
submission_id	INT	NOT NULL, UNIQUE, FK → Submissions(submission_id)
marks	DECIMAL(5,2)	NOT NULL, CHECK ≥ 0
feedback	TEXT	—
graded_by	INT	NOT NULL, FK → Users(user_id)
graded_date	DATETIME	NOT NULL, DEFAULT CURRENT_TIMESTAMP

Grade_Audit

Column	Type	Constraints
audit_id	INT	PRIMARY KEY, AUTO_INCREMENT
grade_id	INT	—
submission_id	INT	—
operation	VARCHAR(10)	INSERT / UPDATE / DELETE
changed_by	VARCHAR(100)	CURRENT_USER()
changed_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP
old_marks	DECIMAL(5,2)	—
new_marks	DECIMAL(5,2)	—
old_feedback	TEXT	—
new_feedback	TEXT	—

Normalization

All tables satisfy **Third Normal Form (3NF)**:

1NF — First Normal Form

All attributes are atomic with no multi-valued or repeating groups. Each cell holds a single, indivisible value. For example, a user's `role` stores exactly one value (`'student'` or `'instructor'`), and `course_code` stores a single unique string per course.

2NF — Second Normal Form

No partial dependencies exist. The `Enrollments` and `Submissions` tables enforce uniqueness via composite unique constraints (`course_id + student_id` and `assignment_id + student_id` respectively), but each table uses a single surrogate primary key, so partial dependency cannot arise. Every non-key attribute depends on the full primary key.

3NF — Third Normal Form

No transitive dependencies exist. Every non-key attribute depends directly and solely on its table's primary key:

- Instructor details (`name`, `email`) are not stored in `Courses`; they are retrieved via JOIN from `Users`.
- Assignment details are not stored in `Submissions`; only the foreign key `assignment_id` is kept for lookup.
- The M:N relationship between students and courses is properly decomposed into the `Enrollments` junction entity — the standard 3NF approach for resolving many-to-many relationships.

SQL Components

Basic Queries (`03_basic_queries.sql`)

20 queries covering the following SQL features:

Category	Features Used
Joins	INNER JOIN, LEFT JOIN across Users, Courses, Enrollments, Assignments, Submissions, Grades
Filtering	WHERE, GROUP BY, HAVING, ORDER BY, LIMIT
Aggregates	COUNT, SUM, AVG, MAX, MIN
String Functions	CONCAT, UPPER, LOWER, LENGTH, SUBSTRING
Date Functions	NOW(), CURDATE(), DATEDIFF(), DATE_FORMAT(), TIMESTAMPDIFF()
NULL Handling	IS NULL, IS NOT NULL, COALESCE, IFNULL

Complex Queries (`04_complex_queries.sql`)

Feature	Description
Subqueries	IN, NOT IN, EXISTS, NOT EXISTS, correlated subqueries
Set Operations	UNION, UNION ALL
WITH Clause (CTE)	Student grade summaries; per-course submission completion statistics
Views	`vw_course_overview`, `vw_student_grades`, `vw_pending_submissions`, `vw_assignment_stats`
Derived Relations	Subqueries in FROM clause
Transaction Control	START TRANSACTION + SAVEPOINT + ROLLBACK demonstration

Stored Procedures & Functions (`05_procedures_functions.sql`)

Name	Type	Purpose
`enroll_student`	Procedure	Enroll a student: validates user role, course existence, and duplicate enrollment; inserts into Enrollments
`submit_assignment`	Procedure	Submit an assignment: verifies student is enrolled, deadline has not passed, and no duplicate submission exists
`grade_submission`	Procedure	Grade a submission: validates marks against `max_marks`, inserts or updates the Grade record
`get_student_avg`	Function	Calculate and return the average marks scored by a given student across all graded submissions
`get_course_stats`	Function	Return total enrollments and submission completion rate for a given course
`is_enrolled`	Function	Return `'YES'` or `'NO'` for whether a student is enrolled in a given course
`cms_procedures`	Group	Bundles: `print_course_summary`, `quick_enroll`, `total_marks_awarded`, `pending_grades`

Cursors

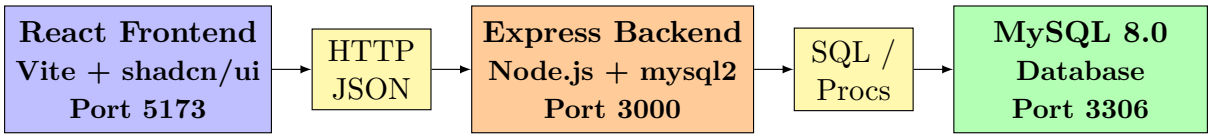
Type	Purpose
Explicit cursor (FETCH loop)	Iterate all submissions for a given assignment; display student name, submission date, and current grade
Parameterised cursor	Fetch all assignments and their submission status for a given `course_id`
Cursor with explicit UPDATE	Bulk-update `Grades` to apply a scaling factor to all marks in a specified course, using the fetched row's primary key

Triggers (06_triggers.sql)

#	Trigger Name	Event	Purpose
1	`trg_check_enrollment`	BEFORE INSERT ON Enrollments	Prevent duplicate enrollment; verify that the user role is `'student'`
2	`trg_check_deadline`	BEFORE INSERT ON Submissions	Reject the submission if the current timestamp exceeds `due_date`
3	`trg_validate_marks`	BEFORE INSERT/UPDATE ON Grades	Ensure `marks` $\leq$ `max_marks` of the associated assignment
4	`trg_audit_grade_changes`	AFTER INSERT/UPDATE/DELETE ON Grades	Log all grade changes to the `Grade_Audit` table
5	`trg_check_enrollment_sub`	BEFORE INSERT ON Submissions	Validate that the submitting student is enrolled in the course
6	`trg_prevent_dup_sub`	BEFORE INSERT ON Submissions	Raise an error if the student has already submitted for the same assignment

System Architecture

The application follows a **three-tier architecture** with strict separation between the presentation, business logic, and data layers:



Frontend (React + Vite + shadcn/ui)

- **Dashboard:** Statistics cards showing total courses, enrolled students, active assignments, and pending grades
- **Courses:** Table with search/filter, course details, and add-course form (instructor only)
- **Users:** Manage students and instructors; view enrollment history dialog
- **Assignments:** Create assignments with deadlines (instructor view); view and submit assignments (student view)
- **Submissions:** Submit work as text content (student view); view and grade submissions (instructor view)
- **Grades:** View grade, feedback, and class statistics per assignment

Backend (Express.js)

- REST API endpoints for all CRUD operations
- Calls MySQL stored procedures for enroll / submit / grade workflows
- JWT-based authentication middleware for role enforcement
- Uses `mysql2` npm package for database connectivity

Database (MySQL 8.0)

- 6 main tables + 1 audit table
- 3 stored procedures, 3 functions, 1 grouped procedure set
- 3 cursor types, 6 triggers, 4 views

Technologies Used

Component	Technology
Database	MySQL 8.0
SQL Dialect	MySQL SQL + Stored Procedures / Triggers
Backend	Node.js + Express.js
DB Driver	mysql2 (npm package)
Authentication	JSON Web Tokens (JWT)
Frontend	React 18 + Vite
UI Library	shadcn/ui + Tailwind CSS
Icons	Lucide React
Routing	React Router v6

Sample Data Summary

Entity	Count	Details
Users	5	3 Students, 2 Instructors
Courses	3	Database Systems (CSS2212), Web Development (CSE3012), Data Structures (CSE2101)
Enrollments	6	Each student enrolled in 2 courses on average
Assignments	5	2 active (within deadline), 2 past-due, 1 due today
Submissions	8	6 on-time, 2 blocked by deadline trigger; 5 graded, 3 pending
Grades	5	Marks range: 45–98; average: 74.6; 3 submissions with written feedback

Conclusion, Limitations and Future Work

Conclusion

The Simple Course Management System successfully demonstrates the practical implementation of database concepts for automating academic workflows. The system manages core functions such as user registration, course enrollment, assignment submission, and grading. By using relational tables with normalisation, SQL queries, stored procedures, functions, and triggers, the project ensures data accuracy, consistency, and reliability.

The database design supports proper relationships among users, courses, assignments, submissions, and grades while preventing invalid operations through constraints and triggers. The integration of a React frontend, Express.js backend, and MySQL database provides a complete working model of a real-world course management application, directly addressing Project 9.3 of *Database System Concepts*.

Limitations

- The system supports only text-based submissions and does not handle file uploads (PDF, DOCX, etc.).
- Discussion or comment threads around specific assignment grades are not yet implemented.
- Poll and course-feedback survey functionality is outside the current scope.
- Fine-grained roles (e.g., teaching assistants vs. instructors) are not differentiated.
- The project uses sample data and may require indexing optimisation for large-scale deployment.

Future Work

- Add file-upload support for assignment submissions (PDF, DOCX, ZIP).
- Implement per-assignment discussion threads for student–instructor interaction.
- Introduce poll and course-feedback mechanisms for gathering student input.
- Add email/SMS notifications for approaching deadlines and grade releases.
- Develop analytics dashboards for class performance, completion rates, and grade distributions.
- Enable cloud deployment for institution-wide, multi-course use.

References

- Silberschatz, A., Korth, H. F., and Sudarshan, S. *Database System Concepts*, 7th Edition. McGraw-Hill Education, 2020. [Project 9.3]
- MySQL 8.0 Reference Manual, Oracle Corporation. `https://dev.mysql.com/doc/`
- Node.js Official Documentation. `https://nodejs.org/en/docs/`
- Express.js Official Documentation. `https://expressjs.com/`
- React Official Documentation. `https://react.dev/`
- Vite Documentation. `https://vitejs.dev/`
- Tailwind CSS Documentation. `https://tailwindcss.com/docs`
- shadcn/ui Component Library. `https://ui.shadcn.com/`
- SQL and course materials provided in Database Systems Lab (CSS 2212), MIT Manipal.